

与上位机(Caesar)对接运控API培训文档

| 适用对象：运运工程师（特别是新入职工程师）。本文档用于了解如何将运控 Dll 提供给上位机(Caesar)，并协助上位机完成 API 调用与程序部署。

| 参考示例：MDA 流程取自 `OptoFidelity.BinoKitefinMDAIQTMTFTTester/UnitTest.cs` 的 `Test()`；FATP 流程取自 `OptoFidelity.BinocRainbowSyetemDisplayEolTester/UnitTest.cs` 的 `Test()`（均为工作区最新版）。

目 录

1. 基础知识点
 - 1.1 运控软件部署与配置文件路径
 - 1.2 不同机型设备点位命名规则
2. 上位机（Caesar）调用运控API流程
 - 2.1 MDA 半成品
 - 2.2 FATP 成品
3. 常用 / 通用 IO 控制 API（BaseTester 基类）
4. 外设模块（相机 / 光源 / Confocal）
5. 提供给上位机（Caesar）的运控Dll列表

一、基础知识点

1.1 运控软件部署与配置文件路径

运控 Dll (`OptoFidelity.*Tester.dll`) 由上位机(Caesar)引用，所有配置与依赖需按以下约定部署。

配置文件路径

路径	说明
D:\MotionConfig\ConfigHardware\	实际运行配置路径（运行时读取，不提交到 Git）
D:\MotionConfig\	配置根目录（ <code>CustomConst.PreConfigHardwarePath</code> ）
OptoMotionLib\doc\Motion机台配置\<机型>\ConfigFile\	项目内默认/参考配置（提交到 Git）

常量定义于 `src/Logic/CustomCore/CustomConst.cs`: `ConfigHardwarePath = @"D:\MotionConfig\ConfigHardware"`。

配置文件目录结构示例

实际运行目录 `D:\MotionConfig\ConfigHardware\` 下的结构如下（以 MDA EVT 实际目录为参考）：

```
D:\MotionConfig\ConfigHardware\
├── ConfigHardware-4059-006\
│   ├── MachineType.json      ← 机台编号目录
│   ├── Axis.json            ← 设备类型（决定 DI 注册哪个 Logic）
│   ├── AxisGroup.json       ← 轴参数
│   ├── MC.json              ← 轴组定义
│   ├── Speed.json           ← 运动控制卡配置
│   ├── MachineSetting.json  ← 速度参数
│   ├── DI.json / DO.json / Cylinder.json ← 机台参数
│   ├── ManagerList.json     ← IO 映射 / 气缸（按需）
│   ├── PointRecipeNameSetting.json ← 外设管理器列表
│   ├── RibbonMenu.xml       ← 点位配方名称映射
│   ├── RibbonMenu-CH.xml    ← 菜单（默认）
│   ├── RibbonMenu-EN.xml    ← 菜单（中文，可选）
│   ├── Point\               ← 菜单（英文，可选）
│   │   ├── Default\         ← 点位配方文件
│   │   ├── Dut138\          ← 默认配方
│   │   ├── Dut143\          ← 产品型号配方
│   │   └── Dut148\
│   ├── cameraConfig.json    ← 相机配置（按需）
│   ├── displacementSensorConfig.json ← 共聚焦传感器配置（按需）
│   ├── lightConfig.json     ← 光源配置（按需）
│   ├── temperatureConfig.json ← 温度配置（按需）
│   └── ComServer.json / Communication.json ← 通信（按需）
```

返回值与错误处理约定

所有运控 API 的返回值遵循以下约定：

返回类型	含义	处理建议
<code>bool</code>	<code>true</code> 成功, <code>false</code> 失败	检查返回值，失败时记录日志

返回类型	含义	处理建议
bool?	true 成功 / false 执行失败 / null 异常错误	需判断三种情况；null 通常为未初始化或总线错误
(bool result, string errMsg)	result 指示成败，errMsg 携带错误描述	result != true 时记录 errMsg，常见错误如 Holder aaCalcTest offset over limit
(bool result, string errMsg, T data)	成功时 data 携带业务数据	先判断 result，再使用 data

建议：每次 API 调用后检查返回值，失败时通过 logger 或 Console.WriteLine 输出错误信息，便于排查。

API 调用约束（可选，根据实际是否需要加锁）

当前 Wrapper 实现中，运动 API 已通过 ExecuteMotionApi() / ExecuteMotionApiReturnBool() 内部加锁。若上位机使用场景涉及多线程调用运动 API，请注意以下约束；单线程顺序调用可忽略。

- 多线程场景：运动 API 内部已加锁，禁止上位机多线程并发调用运动 API。
- 禁止直接调用 motionLogic.xxx()：必须通过 Wrapper 接口调用，确保锁机制生效。
- 初始化顺序：必须先调用 InitializeFixture() 完成初始化（打开运动控制卡、连接外设），之后再调用其他 API。
- 退出前卸载：程序退出前必须调用 FinalizeFixture() 释放资源（关闭运动控制卡、光源、相机等连接）。
- 切换产品配方：切换产品时，先停机 → 调用 ReloadDutTypeSetting() 切换产品类型配置和点位 → MachineHome() 整机复位 → 开始新测试。

配置文件类型说明

每个机型的配置位于 ConfigHardware_<机台编号>\ 目录下（如 ConfigHardware-4059-006\），目录内文件结构如下（以 MDA EVT 实际目录为参考）：

文件	必须	说明
MachineType.json	□	设备类型枚举，决定 DI 注册哪个 MotionLogic（如 BinocKitefinMda、BinocSystemDisplay）
Axis.json	□	轴参数配置（轴数、软硬限位、回零方向等）
AxisGroup.json	□	轴组定义（将物理轴组合为 Holder / Rainbow / Leveling 等逻辑轴组）
MC.json	□	运动控制卡配置
Speed.json	□	速度参数
MachineSetting.json	□	机台参数
DI.json / DO.json	□	数字输入/输出 IO 映射
ManagerList.json	□	外设管理器列表（注册相机/光源/Confocal 等插件）
Point/	□	各机型点位配方文件（子目录按产品型号存放，如 Dut138、Dut143、Dut148，以及 Default）

文件	必须	说明
RibbonMenu.xml	□	菜单配置文件（默认）
RibbonMenu-CH.xml	可选	中文菜单配置
RibbonMenu-EN.xml	可选	英文菜单配置
cameraConfig.json	按需	相机配置（有相机时提供）
displacementSensorConfig.json	按需	共聚焦传感器配置（有 Confocal 时提供）
lightConfig.json	按需	光源控制器配置（Rsee 控制器）
temperatureConfig.json	按需	温度传感器配置
ePRegulatorConfig.json	按需	压力调节器配置（FATP 温度闭环机型）
ComServer.json / Communication.json / Cylinder.json	按需	通信服务、通信协议、气缸配置（视机型需要）

▮ RibbonMenu 加载规则：程序根据当前语言设置依次尝试加载 RibbonMenu-CH.xml（中文）或 RibbonMenu-EN.xml（英文）；若对应语言文件不存在，则回退加载默认的 RibbonMenu.xml。菜单参考模板位于 doc\菜单参考模板\。

▸ 机型配置对照表

▮ 详细配置参见 doc\Motion机台配置\OptoMotionLib 项目配置说明.xlsx（含 feature / dev 分支各机型）。下表为 MDA 与 FATP 两类机型的配置摘要。

项目	MDA（半成品测试）	FATP（成品测试）
设备名称	Norman2(Kitefin) Monoc/Binoc MDA Tester	Norman2(Kitefin) Binoc P1/PP1 System IQT MTF
MotionDll 项目	OptoFidelity.BinocKitefinMDAIQTMTFTester	OptoFidelity.BinocRainbowSyetemDisplayEolTester
MachineType	BinocKitefinMda	BinocSystemDisplay
轴组构成	Holder 轴组 + Leveling Z 轴	Holder 轴组 + 左目 Rainbow XYZ 轴组 + 右目 Rainbow XYZ 轴组
外设	1 × Sszn（或 Keyence）Confocal + 照明光源	温度 + 压力（EPRegulator）
参考配置路径	doc\Motion机台配置\MDA\BinocKitefinMda\ConfigFile\	doc\Motion机台配置\SystemDisplay\ConfigFile\

□ 1.2 不同机型设备点位命名规则

▮ 参考 doc\Motion机台配置\不同机型轴组点位命名.xlsx。点位命名统一采用 PascalCase，按「轴组 + 功能 + 序号」组织。

▸ 轴组分类

按测试类别分为 FATP（成品测试）与 MDA（半成品测试）两大类。

FATP（成品测试）

机型类别	设备构成	轴组	典型 MachineType
单目 WUC	Holder + 左 Rainbow	Holder、 Rainbow	EolFATP_WUC
单目 SystemIQ TMTF	Holder + Rainbow	Holder、 Rainbow	EolFATP_SystemDisplay
双目 SystemIQ TMTF / 双目 WUC	Holder + 左目 Rainbow + 右目 Rainbow	Holder、 LeftEyeRainbow、 RightEyeRainbow	BinocSystemDisplay
SubDisplay	Holder + Rainbow	Holder、 Rainbow	EolFATP_SubDisplay
ST IQT	Holder + Rainbow	Holder、 Rainbow	GtkStlqt
双 Rainbow IQT MTF WPC	Holder + 双 Rainbow + TopVision	Holder、 LeftEyeRainbow、 RightEyeRainbow、 TopVision	GtklqtWpcMtfEx
GDC/PMQ	双 Camera	Camera1、 Camera2	GtkGdcPmq
Artifact	Holder + TopVision	Holder、 TopVision	ArtifactTester

MDA（半成品测试）

机型类别	设备构成	轴组	典型 MachineType
MDA Kitefin	Holder + Leveling	Holder、 Leveling	BinocKitefinMda
MDA Diamond / TW	Holder + Leveling (XYZ)	Holder、 Leveling	GtkMda

› 具体机型点位命名

各机型轴组点位命名详见 [doc\Motion机台配置\不同机型轴组点位命名.xlsx](#)，包含 Holder / Rainbow / LeftEyeRainbow / RightEyeRainbow / Leveling / Camera1 / Camera2 / TopVision 等各轴组的完整点位定义。

› 获取点位的方法

通过基类 [IBaseInterface](#) 获取轴组全部点位：

```
Dictionary<string, double[]> points = wrapper.GetAxisGroupPointList("Holder");
```

或获取点位名称列表（FATP）：

```
List<string> names = wrapper.GetHolderPointNameList();
```

二、上位机 (Caesar) 调用运控API流程

以下流程直接来源于各自 `UnitTest.cs` 的 `Test()` 方法 (工作区最新版) , 作为上位机对接的推荐调用顺序。

2.1 MDA 半成品 (OptoFidelity.BinoKitefinMDAIQTMTFTTester)

Wrapper: `MDA_BinocKitefinWapper`

接口: `IMDA_BinocKitefinWrapper : IBaseInterface`

参考示例: `OptoFidelity.BinoKitefinMDAIQTMTFTTester/UnitTest.cs` -> `Test()`

完整测试流程

序号	API 调用	说明
1	<code>InitializeFixture(dutTypeRecipeName: dutType)</code>	加载载具/初始化运控
2	<code>ReloadDutTypeSetting(dutType)</code>	(可选) 切换产品类型配方
3	<code>MachineHome()</code>	整机复位
4	<code>HolderMoveToLoadUnLoadPos()</code>	载具移动到上下料位置
5	<code>ReadStartBtn_1()</code> / <code>ReadStartBtn_2()</code>	读取双启动按钮状态
6	<code>LightOnOff(false)</code>	关闭照明灯
7	<code>CloseDoor()</code>	关闭上料门
—	左目测试 (isLeftEyeDut = true)	
8	<code>GetAlgoSn(true, snImgDir)</code>	(可选) AA 算法拍照自动识别 SN
9	<code>AaTestWithOffset(true, sn)</code>	AA 测试, 返回六轴偏移量 (mm / deg)
10	<code>ClearAaCalcParam(true, sn)</code>	(可选) 清除 AA offset
11	<code>HolderMoveToSEBTestingPos(true, mpdOffset, ochOffset)</code>	载具移动到 SEB 副眼测试位
12	客户采图 + 图像分析	客户调用 Rainbow 采图 API
13	<code>HolderMoveToTestingPos(true, pointIndex, mpdOffset, ochOffset)</code>	载具移动到 pupil 点位 (pointIndex 1..13 循环, 客户采图)
—	右目测试 (isLeftEyeDut = false)	同左目 8'~13'
14	<code>HolderMoveToLoadUnLoadPos()</code>	载具移动到上下料位置
15	<code>OpenDoor()</code>	打开上料门
16	<code>LightOnOff(true)</code>	打开照明灯
17	<code>FinalizeFixture()</code>	退出前卸载载具

› 上位机调用示例 (MDA)

以下为 `OptoFidelity.BinoKitefinMDAIQTMTFTTester/UnitTest.cs` 中 `Test()` 方法的完整流程，作为上位机对接参考：

```
MDA_BinoKitefinWrapper fixtureWrapper = new MDA_BinoKitefinWrapper();

public bool Test()
{
    try
    {
        // 1.
        var bret = fixtureWrapper.InitializeFixture();
        if (!bret) { Console.WriteLine("Initialize Fixture failed."); return false; }

        //
        string dutType = "Dut138";
        bret = fixtureWrapper.ReloadDutTypeSetting(dutType);
        if (!bret) { Console.WriteLine($"ReloadDutTypeSetting dutType={dutType} failed."); return false; }

        //
        var bret1 = fixtureWrapper.MachineHome();
        if (true != bret1) { Console.WriteLine("MachineHome failed."); return false; }

        // 2. Holder
        var ret = fixtureWrapper.HolderMoveToLoadUnLoadPos();
        if (ret.result != true) { Console.WriteLine("Move to load/unload failed."); return false; }

        // 3.
        bool startSts1 = fixtureWrapper.ReadStartBtn_1();
        bool startSts2 = fixtureWrapper.ReadStartBtn_2();
        bool canStart = startSts1 && startSts2;

        // 4.
        fixtureWrapper.LightOnOff(false);
        fixtureWrapper.CloseDoor();

        // ----      ----
        {
            //
            var isLeftEyeDut = true;
            string sn = "test123Left";

            //      AA      SN
            string snImgDir = ""; //      D:\MotionConfig\AaSn\
            var retSn = fixtureWrapper.GetAlgoSn(isLeftEyeDut, snImgDir);
            if (retSn.result != true) { Console.WriteLine($"GetAlgoSn failed.errMsg-{retSn.errMsg}"); return false; }
            sn = retSn.sn;

            // AA
            var mret = fixtureWrapper.AaTestWithOffset(isLeftEyeDut, sn);
            if (mret.result != true) { Console.WriteLine("AaTestWithOffset failed."); return false; }

            //      AA offset
            fixtureWrapper.ClearAaCalcParam(isLeftEyeDut: true, sn: "test123");

            //      SEB
            double mpdOffset = 0, ochOffset = 0;
            var sebRet = fixtureWrapper.HolderMoveToSEBTestingPos(isLeftEyeDut, mpdOffset, ochOffset);
            if (sebRet.result != true) { Console.WriteLine("Move to SEB failed."); return false; }

            //      +
            //....

            // pupil      pointIndex 1..n
            int pupilNum = 5;
            for (int pointIndex = 1; pointIndex &lt;= pupilNum; pointIndex++)
            {
                var mret1 = fixtureWrapper.HolderMoveToTestingPos(isLeftEyeDut, pointIndex, mpdOffset, ochOffset);
                if (mret1.result != true) { Console.WriteLine($"Move to testing pos {pointIndex} failed."); return false; }
            }
        }
    }
}
```

```

        //      +
    }

    //      isLeftEyeDut = false
    isLeftEyeDut = false;
    sn = "test123Right";
    // ...      4.x'~7'      ...
}

// 8. Holder
ret = fixtureWrapper.HolderMoveToLoadUnLoadPos();
if (ret.result != true) { Console.WriteLine("Move to load/unload failed."); return false; }

// 9.
fixtureWrapper.OpenDoor();
fixtureWrapper.LightOnOff(true);

//
fixtureWrapper.FinalizeFixture();
return true;
}
catch (Exception ex)
{
    Console.WriteLine($"{ex.Message}");
    return false;
}
}

```

› MDA 专属 API 说明

| 以下 API 均为 **IMDA_BinocKitefinWrapper** 相对基类 **IBaseInterface** 的扩展，按功能分类。所有方法均接受可选参数 **Action<string> logger = null**。

1. 载具 (Holder) 运动

API	说明	关键参数
(bool result, string errMsg) HolderMoveToLoadUnLoadPos(...)	载具移动到上下料位置	-
(bool result, ...) HolderMoveToConfocalPos(bool isLeftEyeDut, int pointIndex, ...)	载具移动到 confocal 位置	-
(bool result, ...) HolderMoveToMvCameraPos(bool isLeftEyeDut, ...)	载具移动到拍照位	-
(bool result, ...) HolderRelMoveOffset(bool isLeftEyeDut, double xOff, double yOff, double zOff, double rxOff, double ryOff, double rzOff, ...)	测试区域六轴相对移动	-
(bool result, ...) HolderRelMoveXyzOffset(bool isLeftEyeDut, double xOff, double yOff, double zOff, ...)	测试区域 XYZ 相对移动	-
(bool result, ...) HolderRelMoveYOffset(bool isLeftEyeDut, double yOffset, ...)	测试区域 Y 轴相对移动	-

2. SEB（副眼）测试区域

API	说明	关键参数
(bool result, string errMsg, double[] aaOffset, double[] rxDutOffset) HolderMoveToSEBTestingPos(bool isLeftEyeDut, double mpdOffset, double ochOffset, ...)	载具移动到 SEB（副眼）测试位	mpdOffset = mesMpd - stdMpd, ochOffset = mesOch - stdOch; 返回 aaOffset[0..5](X/Y/Z/Rx/Ry/Rz) 与 rxDutOffset[0..2](X/Y/Z)
(bool result, ...) HolderSEBRelMoveOffset(bool isLeftEyeDut, double xOff, double yOff, double zOff, double rxOff, double ryOff, double rzOff, ...)	SEB 区域六轴相对移动	-
(bool result, ...) HolderSEBRelMoveXyzOffset(bool isLeftEyeDut, double xOff, double yOff, double zOff, ...)	SEB 区域 XYZ 相对移动	-

3. 载具测试区域（pupil 测试）

API	说明	关键参数
(bool result, string errMsg, double[] aaOffset, double[] rxDutOffset) HolderMoveToTestingPos(bool isLeftEyeDut, int pointIndex, double mpdOffset, double ochOffset, ...)	载具移动到 pupil 测试点位	pointIndex 从 1 开始; 超限位报错 Holder aaCalcTest offset over limit

4. Leveling 机构

API	说明	关键参数
(bool result, ...) LevelingMoveToSafePos(...)	Leveling 机构移动到安全零位	-
(bool result, ...) LevelingMoveToMvCameraPos(...)	Leveling 机构移动到拍照位	-

5. AA 算法与温度

API	说明	关键参数
(bool result, string errMsg, string sn) GetAlgoSn(bool isLeftEyeDut, string outputImgDir, ...)	通过 AA 算法拍照自动识别 SN（客户启用算法获取 SN 时调用）	outputImgDir 不指定默认 D:\MotionConfig\AaSn\

API	说明	关键参数
(bool result, double xOffset, double yOffset, double zOffset, double rxOffset, double ryOffset, double rzOffset) AaTestWithOffset(bool isLeftEyeDut, string sn, ...)	AA 测试, 返回六轴偏移量 (mm / deg)	重载可指定 aalmgDir (默认 D:\MotionConfig\AaCalc\)、exposureTimeMs
bool? ClearAaCalcParam(bool isLeftEyeDut, string sn, ...)	清空 AA 测试计算结果 (不需要 AaOffset 时调用)	-
(bool result, string errMsg, double temperature) GetTempertura(int index, ...)	读取温度 (可选, 仅当设备配置温度传感器时可用)	index 从 1 开始, 单位 °C

6. 标定参数与位置状态获取

API	说明	关键参数
(bool result, double calibX, double calibY, double calibZ, double calibRx, double calibRy, double calibRz) GetAaCalibXyzRxRyRz(bool isLeftEyeDut, string sn, ...)	获取 AA 标定 Offset 参数	返回标定结果 X/Y/Z(mm)、Rx/Ry/Rz(deg)
(bool result, double x, double y, double z, double rx, double ry, double rz) GetAaOkLastAlgoParam(bool isLeftEyeDut, string sn, ...)	获取 AA OK 后最后一次计算 XyzRxRyRz	-
(bool result, double x, double y, double z, double rx, double ry, double rz) GetHolderModulePos(...)	获取载具当前六轴坐标	返回 X/Y/Z/Rx/Ry/Rz
(bool result, double z) GetLevelingModulePos(...)	获取 Leveling 当前 Z 坐标	-

□ 2.2 FATP 成品 (OptoFidelity.BinocRainbowSyetemDisplayEolTester)

Wrapper: `FATP_BinocRainbowSystemDispalyEolWrapper`

接口： IFATP_BinocRainbowSystemDispalyEolWrapper : IBaseInterface

参考示例： OptoFidelity.BinocRainbowSyetemDisplayEolTester/UnitTest.cs -> Test()

完整测试流程

序号	API 调用	说明
1	InitializeFixture(dutTypeRecipeName: dutType)	加载载具/初始化（示例产品 Dut148）
2	ReloadDutTypeSetting(dutType)	（可选）切换产品配方（示例切到 Dut138）
3	MachineHome()	整机复位
4	HolderMoveToLoadUnLoadPos()	Holder 移动到上下料位置
5	ReadStartBtn_1() / ReadStartBtn_2()	读取双启动按钮状态（双按钮信号）
6	LightOnOff(false)	关闭照明灯
7	CloseDoor()	关闭上料门
—	Seethrough 测试（左眼）	
8	HolderMoveToLeftEyeSeethroughPosition()	Holder 移动到左眼 Seethrough 测试位
9	RightEyeRainbowMoveToLeftLensSeethroughCenter()	右目 Rainbow 移动到左 Lens Seethrough 中心位
10	RightEyeRainbowMoveToLeftLensSeethroughPosition(pointIndex, mpdOffset, ochOffset)	右目 Rainbow 移动到左 Lens Seethrough 点位（pointIndex 1..3 循环）
11	客户采图 + 图像分析	客户图像分析
12	RightEyeRainbowMoveToLeftLensSeethroughSafePosition()	右目 Rainbow 左 Lens Seethrough 回安全位
—	Seethrough 测试（右眼）	
13	HolderMoveToRightEyeSeethroughPosition()	Holder 移动到右眼 Seethrough 测试位
14	RightEyeRainbowMoveToRightLensSeethroughCenter()	右目 Rainbow 移动到右 Lens Seethrough 中心位
15	RightEyeRainbowMoveToRightLensSeethroughPosition(pointIndex, mpdOffset, ochOffset)	右目 Rainbow 移动到右 Lens Seethrough 点位（pointIndex 1..3 循环）
16	客户采图 + 图像分析	客户图像分析
17	RightEyeRainbowMoveToRightLensSeethroughSafePosition()	右目 Rainbow 右 Lens Seethrough 回安全位
—	Pupil 测试	
18	HolderMoveToPupilTestPos()	Holder 移动到 Pupil 测试位

序号	API 调用	说明
19	<code>LeftEyeRainbowMoveToPupilTestPos(pointIndex, mpdOffset, ochOffset)</code>	左目 Rainbow 移动到 Pupil 点位 (pointIndex 1..5 循环, 客户采图)
20	<code>RightEyeRainbowMoveToPupilTestPos(pointIndex, mpdOffset, ochOffset)</code>	右目 Rainbow 移动到 Pupil 点位 (pointIndex 1..5 循环, 客户采图)
21	<code>LeftEyeRainbowPupilTestMoveToSafetyPos()</code>	左目 Rainbow Pupil 回安全位
22	<code>RightEyeRainbowPupilTestMoveToSafetyPos()</code>	右目 Rainbow Pupil 回安全位
23	<code>HolderMoveToLoadUnLoadPos()</code>	Holder 移动到上下料位置
24	<code>OpenDoor()</code>	打开上料门
25	<code>LightOnOff(true)</code>	打开照明灯 (视工艺需要)
26	<code>FinalizeFixture()</code>	退出前卸载载具

› 上位机调用示例 (FATP)

以下为 `OptoFidelity.BinocRainbowSyetemDisplayEolTester/UnitTest.cs` 中 `Test()` 方法的完整流程, 作为上位机对接参考:

```
FATP_BinocRainbowSystemDispalyEolWrapper fixtureWrapper = new FATP_BinocRainbowSystemDispalyEolWrapper();

public bool Test()
{
    try
    {
        // 1.
        string dutType = "Dut148";
        var bret = fixtureWrapper.InitializeFixture(dutTypeRecipeName: dutType);
        if (!bret) { Console.WriteLine("Initialize Fixture failed."); return false; }

        //
        dutType = "Dut138";
        bret = fixtureWrapper.ReloadDutTypeSetting(dutType);
        if (!bret) { Console.WriteLine($"ReloadDutTypeSetting dutType={dutType} failed."); return false; }

        //
        var bret1 = fixtureWrapper.MachineHome();
        if (true != bret1) { Console.WriteLine("MachineHome failed."); return false; }

        // 2. Holder
        var ret = fixtureWrapper.HolderMoveToLoadUnLoadPos();
        if (ret.result != true) { Console.WriteLine("Move to load/unload failed."); return false; }

        // 3.      +      +
        bool startSts1 = fixtureWrapper.ReadStartBtn_1();
        bool startSts2 = fixtureWrapper.ReadStartBtn_2();
        bool canStart = startSts1 && startSts2;
        fixtureWrapper.LightOnOff(false);
        fixtureWrapper.CloseDoor();

        // ----      ----
        {
            // 4. Seethrough
            ret = fixtureWrapper.HolderMoveToLeftEyeSeethroughPosition();
            if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

            ret = fixtureWrapper.RightEyeRainbowMoveToLeftLensSeethroughCenter();
```

```

        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

        int seethroughPointNum = 3;
        for (int pointIndex = 1; pointIndex &lt;= seethroughPointNum; pointIndex++)
        {
            double mpdOffset = 0, ochOffset = 0;
            var mret1 = fixtureWrapper.RightEyeRainbowMoveToLeftLensSeethroughPosition(pointIndex, mpdOffset, ochOffset);
            if (mret1.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }
            //      +
        }

        ret = fixtureWrapper.RightEyeRainbowMoveToLeftLensSeethroughSafePosition();
        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

        // Seethrough
        ret = fixtureWrapper.HolderMoveToRightEyeSeethroughPosition();
        // ...      RightEyeRainbowMoveToRightLensSeethrough* ...

        // 5. Pupil
        ret = fixtureWrapper.HolderMoveToPupilTestPos();
        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

        int pupilNum = 5;
        for (int pointIndex = 1; pointIndex &lt;= pupilNum; pointIndex++)
        {
            double mpdOffset = 0, ochOffset = 0;
            var mret = fixtureWrapper.LeftEyeRainbowMoveToPupilTestPos(pointIndex, mpdOffset, ochOffset);
            if (mret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }
            //      +

            var mret1 = fixtureWrapper.RightEyeRainbowMoveToPupilTestPos(pointIndex, mpdOffset, ochOffset);
            if (mret1.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }
            //      +
        }

        ret = fixtureWrapper.LeftEyeRainbowPupilTestMoveToSafetyPos();
        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }
        ret = fixtureWrapper.RightEyeRainbowPupilTestMoveToSafetyPos();
        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

        // 6. Holder
        ret = fixtureWrapper.HolderMoveToLoadUnLoadPos();
        if (ret.result != true) { Console.WriteLine($"{ret.errMsg}"); return false; }

        // 7.
        fixtureWrapper.OpenDoor();
        fixtureWrapper.LightOnOff(true);
    }

    //
    fixtureWrapper.FinalizeFixture();
    return true;
}
catch (Exception ex)
{
    return false;
}
}

```

› FATP 专属 API 说明

API	说明	关键参数
(bool result, string errMsg) HolderMoveToLoadUnLoadPos(...)	Holder 移动到上下料位	-
(bool result, string errMsg) HolderMoveToLeftEyeSeethroughPosition(...)	Holder 移动到左眼 Seethrough 测试位	-

API	说明	关键参数
(bool result, string errMsg) HolderMoveToRightEyeSeethroughPosition(...)	Holder 移动到右眼 Seethrough 测试位	-
(bool result, string errMsg) HolderMoveToPupilTestPos(...)	Holder 移动到 Pupil 测试位	-
(bool result, string errMsg) RightEyeRainbowMoveToLeftLensSeethroughCenter(...)	右目 Rainbow 移动到左 Lens Seethrough 中心位	-
(bool result, string errMsg, double[] rxDutOffset) RightEyeRainbowMoveToLeftLensSeethroughPosition(int pointIndex, double mpdOffset, double ochOffset, ...)	右目 Rainbow 移动到左 Lens Seethrough 点位	pointIndex 从 1 开始; mpdOffset/ochOffset = 当前产品 - Normal 产品差值
(bool result, string errMsg) RightEyeRainbowMoveToLeftLensSeethroughSafePosition(...)	右目 Rainbow 左 Lens Seethrough 回安全位	-
(bool result, string errMsg) RightEyeRainbowMoveToRightLensSeethroughCenter(...)	右目 Rainbow 移动到右 Lens Seethrough 中心位	-
(bool result, string errMsg, double[] rxDutOffset) RightEyeRainbowMoveToRightLensSeethroughPosition(int pointIndex, double mpdOffset, double ochOffset, ...)	右目 Rainbow 移动到右 Lens Seethrough 点位	同上
(bool result, string errMsg) RightEyeRainbowMoveToRightLensSeethroughSafePosition(...)	右目 Rainbow 右 Lens Seethrough 回安全位	-
(bool result, string errMsg, double[] rxOffset) LeftEyeRainbowMoveToPupilTestPos(int pointIndex, double mpdOffset, double ochOffset, ...)	左目 Rainbow -> Pupil 测试位	-
(bool result, string errMsg, double[] rxOffset) RightEyeRainbowMoveToPupilTestPos(int pointIndex, double mpdOffset, double ochOffset, ...)	右目 Rainbow -> Pupil 测试位	-
(bool result, string errMsg) LeftEyeRainbowPupilTestMoveToSafetyPos(...)	左目 Rainbow Pupil 回安全位	-
(bool result, string errMsg) RightEyeRainbowPupilTestMoveToSafetyPos(...)	右目 Rainbow Pupil 回安全位	-
(bool result, string errMsg) LeftEyeRainbowPupilSwimming(double xOff, double yOff, double zOff, ...)	左目 Rainbow Pupil swimming 偏移移动	-
(bool result, string errMsg) RightEyeRainbowPupilSwimming(...)	右目 Rainbow Pupil swimming 偏移移动	-
(bool result, string errMsg) RainbowMoveToPupilReadyPosition(bool isLeftEye, ...)	Rainbow 移动到 Pupil 就绪位	isLeftEye: true 左目
(bool result, string errMsg, double x, double y, double z, double rx, double ry, double rz) GetHolderPosition(...)	获取 Holder 六轴当前位置	-
(bool result, string errMsg, double x, double y, double z) GetLeftEyeRainbowPosition(...)	获取左目 Rainbow 当前位置	-

API	说明	关键参数
(bool result, string errMsg, double x, double y, double z) GetRightEyeRainbowPosition(...)	获取右目 Rainbow 当前位置	-
string GetAllAxisPositionToString(...)	获取所有轴组当前位置格式化字符串	-

三、常用 / 通用 IO 控制 API (基类 BaseTester)

以下 API 来自 IBaseInterface (OptoFidelity.BaseTester)，两类机型通用。所有方法均接受可选参数 Action<string> logger = null。

□ 3.1 设备生命周期

API	说明	返回值
bool InitializeFixture(Action<string> logger = null, string dutTypeRecipeName = "Default")	加载工装，初始化运控（打开运动控制卡、连接外设等）	true 成功
void FinalizeFixture(Action<string> logger = null)	卸载工装，关闭运动控制卡、光源、相机、激光测距仪等连接	无
bool ReloadDutTypeSetting(string dutTypeRecipeName, Action<string> logger = null)	重载产品类型及参数（切换配方）	true 成功
bool? FixtureClearErr(Action<string> logger = null)	清除运动控制卡总线错误和轴报警	true 成功 / false 失败 / null 异常
bool? GetMachineHomeStatus(Action<string> logger = null)	获取设备整机复位状态	true 已复位 / false 未复位 / null 异常
bool? MachineHome(Action<string> logger = null)	整机复位（所有电机按顺序回原）	true 成功 / false 失败 / null 异常

□ 3.2 产品检测

API	说明	返回值
<code>bool? IsDutExit(Action<string> logger = null)</code>	检查产品是否在载具	<code>true</code> 有产品 / <code>false</code> 无产品 / <code>null</code> 异常

□ 3.3 门控制

API	说明	返回值
<code>bool? OpenDoor(Action<string> logger = null)</code>	打开上料门	<code>true</code> 成功 / <code>false</code> 失败 / <code>null</code> 异常
<code>bool? CloseDoor(Action<string> logger = null)</code>	关闭上料门	同上
<code>bool? DoorLock(bool on, Action<string> logger = null)</code>	安全门锁控制 (<code>true</code> 锁 / <code>false</code> 解锁)	同上

□ 3.4 按钮状态读取

API	说明	返回值
<code>bool ReadStartBtn(Action<string> logger = null)</code>	读取 Start 按钮状态	<code>true</code> 按下 / <code>false</code> 未按下
<code>bool ReadStartBtn(int index, Action<string> logger = null)</code>	通过索引读取 Start 按钮 (从 1 开始)	同上
<code>bool ReadStartBtn_1(Action<string> logger = null)</code>	读取 Start1 按钮	同上
<code>bool ReadStartBtn_2(Action<string> logger = null)</code>	读取 Start2 按钮	同上
<code>bool ReadResetBtn(Action<string> logger = null)</code>	读取 Reset 按钮	同上
<code>bool ReadEmgStopBtn(Action<string> logger = null)</code>	读取急停按钮	同上
<code>bool ReadStopBtn(Action<string> logger = null)</code>	读取停止按钮	同上

□ 3.5 光栅 (安全光幕)

API	说明	返回值
<code>bool ReadGratingStatus(Action<string> logger = null)</code>	读取光栅状态	<code>true</code> 被触发 / <code>false</code> 安全

API	说明	返回值
<code>bool ReadGratingStatus(int index, Action<string> logger = null)</code>	通过索引读取光栅 (从 1 开始)	同上
<code>bool ReadGrating1Status(Action<string> logger = null)</code>	读取光栅 1	同上
<code>bool ReadGrating2Status(Action<string> logger = null)</code>	读取光栅 2	同上

□ 3.6 三色灯 / 五色灯

API	说明
<code>bool? TriColorLightRed(bool on, ...)</code>	打开/关闭三色灯 红灯
<code>bool? TriColorLightGreen(bool on, ...)</code>	打开/关闭三色灯 绿灯
<code>bool? TriColorLightYellow(bool on, ...)</code>	打开/关闭三色灯 黄灯
<code>bool? TriColorLightBlue(bool on, ...)</code>	打开/关闭三色灯 蓝灯
<code>bool? TriColorLightWhite(bool on, ...)</code>	打开/关闭三色灯 白灯

参数 `on`: `true` 打开, `false` 关闭。

□ 3.7 蜂鸣器 / 按钮灯

API	说明
<code>bool? Buzzer(bool on, ...)</code>	打开/关闭蜂鸣器
<code>bool? ButtonLightStart(bool on, ...)</code>	打开/关闭 Start 按钮灯
<code>bool? ButtonLightStart(int index, bool on, ...)</code>	通过索引打开/关闭启动按钮灯 (从 1 开始)
<code>bool? ButtonLightStart1(bool on, ...)</code>	打开/关闭 Start1 按钮灯
<code>bool? ButtonLightStart2(bool on, ...)</code>	打开/关闭 Start2 按钮灯
<code>bool? ButtonLightStop(bool on, ...)</code>	打开/关闭停止按钮灯
<code>bool? ButtonLightReset(bool on, ...)</code>	打开/关闭复位按钮灯

□ 3.8 照明灯

API	说明
<code>bool? LightOnOff(bool on, ...)</code>	控制单个照明灯亮灭 (<code>true</code> 开 / <code>false</code> 关)
<code>bool? LightOnOff(int index, bool on, ...)</code>	通过索引控制多个照明灯 (从 1 开始)

□ 3.9 PCB / Hub 供电

API	说明
<code>bool? PcbPowerOnOff(bool on, ...)</code>	PCB 板 / Hub 供电控制 (<code>true</code> 上电 / <code>false</code> 掉电)
<code>bool? PcbPowerOnOff(int index, bool on, ...)</code>	多个 Pcb / Hub 供电通断 (从 1 开始)

□ 3.10 IO 读写 (通用 DI / DO)

API	说明
<code>bool? ReadDi(int index, out bool status, ...)</code>	通过索引读取输入 IO 状态
<code>bool? ReadDi(string diName, out bool status, ...)</code>	通过名称读取输入 IO 状态
<code>bool? ReadDo(int index, out bool status, ...)</code>	通过索引读取输出 IO 状态
<code>bool? ReadDo(string doName, out bool status, ...)</code>	通过名称读取输出 IO 状态
<code>bool? WriteDo(int index, bool value, ...)</code>	通过索引写输出 IO
<code>bool? WriteDo(string doName, bool status, ...)</code>	通过名称写输出 IO

□ 3.11 通用运动 API

API	说明
<code>Dictionary<string, double[]> GetAxisGroupPointList(string groupName, ...)</code>	获取轴组所有点位 (pointName -> pos 字典)
<code>(bool result, string errMsg) MoveAxisRel(string groupName, int axisId, double offset, ...)</code>	单轴相对移动 (<code>axisId</code> 0-5 -> X,Y,Z,Rx,Ry,Rz)
<code>(bool result, string errMsg) AxisGroupMoveRel(string groupName, int[] axisIdList, double[] offset, ...)</code>	轴组多轴相对移动

四、外设模块 (相机 / 光源 / Confocal)

| 重要说明: 相机、光源、Confocal 不在 Wrapper 接口中直接暴露, 而是 DLL 内部通过依赖注入 (DI) + VisionTools 插件架构, 在 `AaTest` / `HolderMoveToConfocalPos` / `Rainbow` 采图等测试流程 API 中自动调用。上位机只需在 `ManagerList.json` 中注册对应插件并编写配置文件即可。

□ 4.1 外设插件架构

外设	接口 (Custom Core)	管理类	插件 DLL	硬件接口	说明
相机	<code>IMvCamera</code> / <code>CameraDevDic</code>	<code>CameraManager</code>	<code>Plugin.Camera.Galaxy.dll</code>	USB3.0 / GigE (大恒 Galaxy Gx 系列)	<code>EnumCamera.Gx = 1</code> , 参考依赖: <code>GxIAP.dll</code> 等

外设	接口 (Custom Core)	管理类	插件 DLL	硬件接口	说明
光源	ILight / LightLDevDic	LightManager	Plugin.Light.RseeController.dll	RS-232 串口 (COM)	EnumLightController.Rsee = 1, 波特率 19200, 参考依赖: RseeController.dll
Confocal (Keyence)	IDisplacementSensor / ConfocalDevDic	DispSensorManager	Plugin.DisplacementSensorController.Keyence.dll	以太网 TCP/IP	EnumDispSensor.Keyence = 1, 参考依赖: CL3_IF.dll
Confocal (Sszn)	IDisplacementSensor / ConfocalDevDic	DispSensorManager	Plugin.DisplacementSensorController.Sszn.dll	以太网 TCP/IP	EnumDispSensor.Sszn = 2, 参考依赖: SGIFPJ.dll
温度	ITemperatureBaseController / TemperatureDevDic	TemperatureManager	Plugin.TemperatureSensor.dll	串口 / Modbus (视型号)	EnumTemperature (1=红外测温仪, 2=LAS_LAD24 热电偶)
压力	IEPRegulator / EPRegulatorDev	EPRegulatorManager	Plugin.EPRegulator.dll	RS-485 / Modbus RTU	FATP 温度闭环压力控制
光谱仪	ISpectrometer / SpectrometerDev	SpectrometerManager	Plugin.Spectrometer.dll	USB	Rainbow 光谱采集

□ 4.2 外设配置 (在 ManagerList.json 中注册)

参考模板: doc/Motion机台配置/外设参考模板/ManagerList-All.json

MDA 配置示例 (含相机/光源/Confocal/温度) :

```
[
  { "ManagerName": "LightManager",      "ConfigFileName": "lightConfig.json",      "PluginType": "lightConfig.json" },
  { "ManagerName": "DispSensorManager", "ConfigFileName": "displacementSensorConfig.json", "PluginType": "displacementSensorConfig.json" },
  { "ManagerName": "CameraManager",     "ConfigFileName": "cameraConfig.json",     "PluginType": "cameraConfig.json" },
  { "ManagerName": "TemperatureManager", "ConfigFileName": "temperatureConfig.json", "PluginType": "temperatureConfig.json" }
]
```

FATP 配置示例 (温度 + 压力) :

```
[
  { "ManagerName": "TemperatureManager", "ConfigFileName": "temperatureConfig.json", "PluginType": "temperatureConfig.json" },
  { "ManagerName": "EPRegulatorManager", "ConfigFileName": "ePRegulatorConfig.json", "PluginType": "ePRegulatorConfig.json" }
]
```

□ 4.3 配置文件字段说明

- lightConfig.json: Channel、Brightness、CotrollerName(Rsee)、PortName(COM)、BaudRate、Name(如 MvCameraLight)
- displacementSensorConfig.json: Ip、Port、DeviceId、Scale、Name(Keyence 或 Sszn)、Channel、IsUseOpticalHeadProbe
- cameraConfig.json: CameraID、Gain、ExposureTimeMs、TimeOut、Name(Gx)
- temperatureConfig.json: EnumTemperature (1=红外测温仪, 2=LAS_LAD24 热电偶)
- ePRegulatorConfig.json: 压力传感器索引 1-2, 通过 SetPressure / GetOutputPressure 读写

□ 4.4 外设相关运控 API

- MDA 温度读取: (bool result, string errMsg, double temperature) GetTempertura(int index, ...)
 - FATP 温度读取: (bool result, string errMsg, double temperature) GetTemperature(ushort channel, ...) (通道 1-8)
 - FATP 温度偏移: (bool result, string errMsg) SetOffset(double offset, ...) (°C)
 - FATP 压力写入: (bool result, string errMsg) SetPressure(int index, double pressure, int timeoutMs=500, ...) (MPa, index 1-2)
 - FATP 压力读取: (bool result, string errMsg, ushort outputValue, double pressure) GetOutputPressure(int index, ...)
 - FATP 通用指令: (bool result, string errMsg, string response) SetGetCommand(int index, string command, ...)
 - MDA 标定参数: (bool result, double calibX...calibRz) GetAaCalibXyzRxRyRz(...) / (bool result, ...) GetAaOkLastAlgoParam(...)
-

■ 五、提供给上位机 (Caesar) 的运控DII列表

| 以下为两类 DII 的 CopyDependency.txt 内容, 部署时需将所列 DII 拷贝至上位机执行目录。

□ 5.1 MDA DII 列表 (OptoFidelity.BinoKitefinMDAIQTMTFTTester)

_相机依赖 (大恒 Galaxy Gx 系列) _

- AxNICCf.dll - DxImageProc.dll - GCBBase_MD_VC120_v3_0.dll - GenApi_MD_VC120_v3_0.dll
- GxIAP.dll - GxIAPICPP.dll - GxIAPICPPEX.dll - GxIAPINET.dll
- mfc90.dll - Microsoft.VC90.CRT.manifest - Microsoft.VC90.MFC.manifest
- msvcp100.dll - msvcp120.dll - msvcp90.dll - msvcr100.dll - msvcr120.dll - msvcr90.dll

Keyence 激光测距仪依赖

- CL3_IF.dll

Sszn 激光测距仪依赖

- SGIFPJ.dll

_光源依赖 (Rsee 控制器) _

- RseeController.dll

日志依赖

- log4net.dll

基类 DII

- OptoFidelity.BaseTester.dll

_运动控制卡依赖 (雷赛) _

- LTSMC.dll

□ 5.2 FATP Dll 列表 (OptoFidelity.BinocRainbowSyetemDisplayEolTester)

- log4net.dll
- OptoFidelity.BaseTester.dll
- LTSMC.dll

| 配置文件路径常量: CustomConst.ConfigHardwarePath =
D:\MotionConfig\ConfigHardware\ (实际运行路径) 。参考配置模板位于
doc/Motion机台配置/MDA/BinocKitefinMda/ConfigFile/ (MDA) 与
doc/Motion机台配置/SystemDisplay/ConfigFile/ (FATP) 。